PYTHON / CONCEPT NOTES

Python 知识点笔记

通俗版

是什么 · 有什么用 · 容易混淆在哪里

从基础到应用的知识路线

01　先理解数据和流程
变量与类型 → 容器 → 判断循环 → 函数与参数

↓

02　再理解程序怎样组织
迭代与生成器 → 函数工具 → 模块与类 → 异常与测试

↓

03　把知识用于实际任务
文件与标准库 → 网络 → 数据库 → Web 与异步

重点　黄色标记需要记住的规则；红色标记易错区别。颜色旁都有文字标签，方便打印阅读。

阅读层级　“基础优先”先弄懂；“进阶理解”和“应用认识”先知道用途，随后按需要深入。

基于廖雪峰《Python 教程》原全目录范围，重写为 27 个知识主题。

原教程作者：廖雪峰；本版重组结构并改写解释，另补充框架概念。

修订日期：2026-11-10 | CC BY-NC-SA 4.0

知识目录

01 先弄清 Python 怎样运行 . . . 3
02 变量、数值和真假判断 . . . 4
03 字符串与字符编码 . . . 5
04 四种常用容器怎么选 . . . 6
05 条件、循环与模式匹配 . . . 7
06 函数、返回值与作用域 . . . 8
07 函数参数与递归 . . . 9
08 切片、迭代与生成器 . . . 10
09 高阶函数、筛选与排序 . . . 11
10 闭包、装饰器与偏函数 . . . 12
11 模块、包与虚拟环境 . . . 13
12 类、对象与面向对象思路 . . . 14
13 面向对象的高级工具 . . . 15
14 异常、调试与测试 . . . 16
15 文件、路径与序列化 . . . 17
16 进程、线程与并发 . . . 18
17 正则表达式 . . . 19
18 时间、数据工具与命令行参数 . . . 20
19 二进制、摘要与资源管理 . . . 21
20 获取网页与解析数据 . . . 22
21 第三方库与图形界面 . . . 23
22 网络通信与电子邮件 . . . 24
23 数据库、SQL 与 ORM . . . 25
24 HTTP、网页与服务端接口 . . . 26
25 后端框架应该怎样理解 . . . 27
26 协程、asyncio 与 aiohttp . . . 28
27 常用排查信息与概念速查 . . . 29
来源与章节对照 . . . 30

01 先弄清 Python 怎样运行

基础优先

原教程第 1—4 章：简介、历史、安装、解释器、第一个程序、输入输出。

Python、解释器和编辑器各做什么

Python 是一门语言，规定程序应该怎样写。**解释器**负责理解并执行程序；常用的 Python 实现是 CPython。**编辑器**帮助你输入和修改代码，本身并不等于 Python。遇到运行问题时，先分清是代码错误、解释器选择错误，还是编辑器设置问题。

Python 2 和 Python 3 存在不兼容之处。现在学习本教程，应以 Python 3 为准；旧文章中的写法不能直接当作当前写法。

两种运行方式

交互模式是输入一小段、马上看结果，适合临时查看和验证。**脚本模式**是把程序保存在 `.py` 文件中，再让解释器运行，适合保存完整程序。

交互环境显示的 `>>>` 和 `...` 是提示符，不属于代码。终端是输入系统命令的地方；Python 交互环境是输入 Python 语句的地方，两者不能混用。

代码的基本外形

Python 用**缩进**区分代码属于哪一层，通常统一使用四个空格。同一代码块的缩进应一致。注释用于说明代码，通常以 `#` 开头，不作为普通语句执行。

名字区分大小写。关键字具有固定用途，不能随意拿来当变量名。源文件通常使用 UTF-8 编码保存。

输入与输出

`print()` 把内容显示出来；`input()` 接收键盘输入。**输入看起来像数字，接收到的仍然是字符串**，需要计算时再转换为合适的数值类型。

`int()` 用于整数转换，`float()` 用于浮点数转换。转换要求输入内容符合相应格式，不符合时会出现异常。

> **重点** 编辑器负责写，解释器负责运行；终端命令和 Python 语句分开看；输入得到的字符串不会自动变成数字。

02 变量、数值和真假判断

基础优先

原教程 5.1：数据类型和变量；结合基础语法整理。

变量可以理解为对象的名字

对象是程序中实际存在的数据；**变量名**是用来找到对象的名字。赋值让名字关联到对象，不是自动复制一份对象。

同一个名字可以先后关联不同类型的对象，这体现了 Python 的动态类型。**类型属于对象**；动态类型并不意味着程序可以随意混合任何类型。

最常用的基本类型

类型	通俗理解	要留意什么
int	表示整数	能处理很大的整数，但仍受内存限制
float	表示带小数的数值	精度有限，计算可能有舍入误差
bool	表示真或假	只有 True 和 False 两个值
None	表示没有值或尚未提供值	不等于数字零，也不等于空字符串

运算符分别表示什么

`+`、`-`、`*` 表示加、减、乘，`**` 表示乘方。`/` 表示普通除法，结果通常是浮点数；`//` 表示向下取整的除法；`%` 表示取余。

“向下取整”是向负无穷方向取整，遇到负数时，不能简单理解为删掉小数部分。除数不能为零。

三种容易混淆的比较

`=` 用于赋值；`==` 比较值是否相等；`is` 判断是不是同一个对象。通常比较数据内容用 `==`，检查是否为 None 用 `is None`。

全大写变量名通常表示常量，但这主要是命名约定，不会自动阻止重新赋值。

逻辑运算会提前停止

`and` 表示“并且”，`or` 表示“或者”，`not` 表示取反。and 和 or 在已经能确定结果时，会跳过后面的计算，这叫**短路**；它们返回的可能是某个参与运算的值，不一定是 bool。

重点 先分清“名字变了”和“对象被修改了”。两个名字可能关联同一个对象，修改共享的可变对象会互相影响。

03 字符串与字符编码

基础优先

原教程 5.2：字符串和编码。

字符串表示文字

字符串 str 用于保存文字，由引号包围。单引号和双引号都可以，三引号还能方便地写多行文字。引号通常是字符串的边界，不属于文字内容。

字符串有顺序，可以按位置读取字符，也可以通过切片取出一部分。**字符串不可变**，转换大小写、替换文字等操作通常得到新字符串。

转义与原始字符串

转义字符用反斜杠表示特殊含义，常见的有 `\n` 换行、`\t` 制表、`\\` 反斜杠本身。

在字符串前加 `r`，通常表示按原样保留反斜杠，适合正则表达式等内容。原始字符串仍受引号规则限制，不能以奇数个反斜杠结束。

文本和字节是两种东西

str 表示文字，bytes 表示字节数据。文字方便人理解；文件和网络传输最终需要处理字节。Unicode 是表示文字的一套统一规则；UTF-8 是把文字转换为字节的一种编码方式。

编码 encode：文字变成字节。**解码 decode**：字节按指定规则还原成文字。解码规则不匹配，可能出现乱码或异常。

长度与格式化

`len()` 可以取得长度，但字符串长度和编码后的字节长度不一定相同。更复杂的组合字符中，码点个数也未必等于肉眼看到的字形个数。

格式化是把变量的值放进一段文字中，并控制显示方式。f-string、`format()` 和百分号格式化都能完成这件事；初学时优先熟悉 f-string 的花括号和格式说明。

重点 记住两个方向：encode 是“文字到字节”，decode 是“字节到文字”。普通文本文件优先明确使用 UTF-8。

易错 “忽略解码错误”可能丢掉内容，不能当作处理乱码的通用办法。应先查清数据实际采用的编码。

04 四种常用容器怎么选

基础优先

原教程 5.3 使用 list 和 tuple、5.7 使用 dict 和 set。

容器用来放一组数据

选择容器时，先想清楚：是否需要顺序，是否需要修改，是否按名字查找，是否允许重复。不要只记住括号长什么样。

容器	核心特点	常见用途
list 列表	有顺序，可以修改，允许重复	保存需要增删和调整的一组数据
tuple 元组	有顺序，各位置的引用不能修改	表示固定结构的一组值
dict 字典	通过键查找值，键不能重复	表示字段和对应的数据
set 集合	元素不重复，不保证排序	去重、判断成员、做集合运算

列表与元组

索引从零开始，负索引从末尾计数。列表可追加、插入、删除元素；元组不能直接替换其中某个位置。**单元素元组要靠逗号区分**，只有括号不一定是元组。

元组不能改的是各位置所关联的对象；如果其中放着可变对象，该对象内部仍可能被修改。

字典与集合

字典通过键找值；`in` 默认判断字典中有没有这个键。`get()` 可在键不存在时返回默认值。现代 Python 的字典保留插入顺序，但这不等于自动排序。

集合可做交集、并集和差集。空字典用 `{}`；空集合用 `set()`。字典的键和集合的元素必须**可哈希**，也就是满足稳定计算哈希值等要求；列表不能直接放在这些位置。

赋值不等于复制

两个名字可能关联同一个列表。**浅复制**只新建外层容器，内部嵌套对象仍可能共享；**深复制**尝试把内部对象也一并复制，适合需要分离嵌套数据的场景。

重点 需要一串数据选 list；固定组合考虑 tuple；需要按键查找选 dict；需要去重或集合关系选 set。

05 条件、循环与模式匹配

基础优先

原教程 5.4 条件判断、5.5 模式匹配、5.6 循环。

条件判断让程序选择一条路

`if` 判断第一个条件，`elif` 继续检查其他条件，`else` 处理其余情况。条件按从上到下的顺序检查，**进入第一个满足的分支后，就不会再检查后面的分支**。

判断范围时，要明确边界是否包含，并让分支顺序符合范围关系。条件重叠时，前面的分支可能挡住后面的分支。

哪些值会被当作假

False、None、数值零、空字符串和空容器等，在条件中通常为假。自定义对象也可以通过相关方法决定真假。

空值、零、空容器是不同概念。如果只想判断有没有提供值，通常应明确检查 None，而不是把所有假值混在一起。

两种循环各适合什么

for 用来逐项处理一组数据，不一定需要数字下标。**while** 在某个条件持续成立时重复执行，适合次数事先不确定的流程。

`range()` 表示一段整数范围：起点包含，终点不包含，步长不能为零。它不会提前建立包含所有元素的普通列表。

怎样改变循环流程

`break` 退出当前最近一层循环；`continue` 跳过本轮剩余语句，开始下一轮。while 中要保证影响条件的状态能够变化，否则可能一直运行。

`pass` 只是空语句，不表示跳过本轮，也不表示退出。

match 主要匹配数据的结构

`match/case` 可按照固定值、序列形状等选择分支；守卫条件可以进一步限制匹配。`case _` 常用于兜底，应放在后面。

模式中的普通名字往往会接收匹配到的值，不能一律当作“拿现有变量来比较”。match 语法需要 Python 3.10 或更高版本。

重点 if 管选择，for 管逐项处理，while 管按条件重复。先把这三种流程想清楚，再考虑更复杂的写法。

06 函数、返回值与作用域

基础优先

原教程第 6 章、6.1 调用函数、6.2 定义函数。

函数是可以反复调用的处理步骤

把一组相关操作放在函数里，可以减少重复，也让程序更容易读。**定义函数**是说明要做什么；**调用函数**才是要求执行这组操作。

参数是调用者交给函数的数据；函数名用于找到这个函数。调用时要遵守参数数量、名字和取值要求。

返回值与打印的区别

return 把结果交回调用者，并结束本次函数调用。**print 把内容显示出来**，不能代替把计算结果返回给后续程序。

函数没有显式返回结果，或者只写了没有值的 return 时，返回 None。所谓“返回多个值”，通常是把多个值组合成一个元组后返回。

名字有自己的可见范围

局部变量属于当前函数的这一次调用；**全局变量**通常定义在模块层面。查找名字时，一般先找当前函数，再找外层函数、模块和内建名字，这称为 LEGB 查找顺序。

函数内部给一个名字赋值，通常会建立局部绑定。要修改外层函数的绑定，可使用 `nonlocal`；要重新绑定模块级名字，可使用 `global`。初学时尽量让数据通过参数和返回值流动。

参数并不会自动隔离对象

调用函数时，传入的是对象的关联。函数若修改传入的可变对象，调用者可能看到变化；如果只把局部参数名改为关联另一个对象，通常不会改掉调用者原有名字的绑定。

这是“修改对象”和“重新赋值”的区别，也是理解列表参数的关键。

重点 理解函数时始终跟踪三件事：输入了什么、内部改变了什么、最终返回了什么。

易错 函数对象本身与函数调用结果不是同一回事。需要把处理方法交给别的函数时，传的是函数；需要结果时，才执行调用。

07 函数参数与递归

基础优先；递归属于进一步理解

原教程 6.3 函数的参数、6.4 递归函数。

各种参数分别解决什么问题

参数形式	用途
位置参数	按传入顺序对应到参数
关键字传参	调用时明确写出参数名，提高可读性
默认参数	调用者省略时，使用已设定的默认值
*args	收集额外的位置参数，得到元组
仅限关键字参数	要求调用者使用参数名来传入
**kwar	收集额外的关键字参数，得到字典

定义时收集，调用时展开

在定义函数的参数列表里，星号常用于**收集参数**；在调用函数时，星号常用于**展开已有数据**。一个星号通常展开位置参数，两个星号展开关键字参数。

同一个参数不能被重复赋值；关键字名必须符合函数接口要求。参数组合越多，接口越难理解，应优先设计简单清楚的函数。

默认参数只在定义时计算

默认值不是每次调用都重新创建。若把可变列表或字典直接作为默认值，多次调用可能共享同一个对象，产生意外影响。

需要每次调用创建新容器时，常用 None 表示“没有传入”，再在函数体中创建容器。这样也能区分“未提供”和“明确传入空容器”。

递归是函数调用自己

递归把当前问题拆成更小的同类问题。它需要**终止条件**，也需要每一步都朝终止条件靠近，否则会不断继续调用。

每次调用会占用一定的调用栈空间，过深时可能触发递归深度异常。常见 CPython 不保证尾递归优化；较深、较长的重复流程可以考虑循环或显式栈。

重点 参数先明确“哪些必填、哪些可省略、哪些必须写名字”。递归先明确“什么时候停、问题怎样变小”。

08 切片、迭代与生成器

切片和迭代优先；生成器随后理解

原教程第 7 章、7.1—7.5。

切片是按范围取出一部分

切片包含起点、不包含终点，可以指定步长；省略部分边界时会使用默认值。单个索引越界通常报错，切片超出范围通常会被截断。

常见列表切片产生新列表，但这通常只是浅复制。步长不能为零；负步长会改变读取方向。

迭代是逐个取得数据

迭代不一定依赖数字下标。列表、字符串、字典、文件等都可以逐项处理。遍历字典默认得到键；需要值或键值对时，分别使用对应接口。

`enumerate()` 同时提供序号和元素；`zip()` 把多个输入按位置配对，默认在最短输入耗尽时结束。

生成式把“取、筛、变”写在一起

列表生成式把遍历、筛选和转换集中表达，结果是已经生成的列表。筛选决定要不要保留元素；条件表达式决定保留元素后输出哪种值，这两种作用不要混淆。

逻辑太复杂时，普通循环通常更易读，不必为了简短而强行挤在一行。

三个相关概念

名词	通俗理解
可迭代对象 Iterable	能提供一套逐项读取的方法
迭代器 Iterator	记录当前读到哪里，可以继续取下一项
生成器 generator	用生成器表达式或 yield 创建的一类迭代器

生成器按需要计算

生成器不必提前算出所有结果，而是在需要下一项时继续执行。`yield` 交出一个值，并暂停在当前位置；下次读取时从这里继续。结束时会产生 StopIteration，for 循环会处理正常结束。

同一个生成器通常只能消费一轮；已经读取的内容不会自动从头再来。列表能反复遍历，是因为可以重新取得新的迭代器。

重点 列表偏向“先准备好”，生成器偏向“需要时再算”。节省内存不等于计算完全没有成本。

09 高阶函数、筛选与排序

基础熟悉后掌握

原教程 8 函数式编程、8.1 高阶函数、8.1.1—8.1.3。

函数本身也能作为数据传递

函数可以关联到变量名，可以传给另一个函数，也可以被函数返回。**高阶函数**就是接收函数作为参数，或者返回函数的函数。

这样可以把“怎样遍历”和“每个元素怎样处理”分开。调用者提供处理规则，通用函数负责执行流程。

map、filter、reduce 的职责

工具	核心作用	结果特点
map	对每个元素应用转换规则	按项转换，通常不改变项数
filter	按条件保留符合要求的元素	可能减少元素数量
reduce	把前面的累计结果与下一项继续合并	最终得到一个累计结果

map 和 filter 返回惰性的迭代器，需要读取时才逐项处理。reduce 来自 functools；空输入时是否能产生结果，与是否提供初始值等约定有关。

排序先确定依据，再确定方向

`sorted()` 返回排序后的新列表；列表的 `sort()` 在原列表上排序，并返回 None。`key` 指定“拿什么当排序依据”，不是让你写两个元素之间的比较过程。

`reverse` 控制排序方向。**稳定排序**指排序依据相同的元素，保持它们原来相互之间的先后顺序。多个排序要求应明确优先级。

匿名函数适合短规则

`lambda` 用一个表达式定义小函数，适合短的转换、判断和排序依据。它不能像普通函数体那样放多条语句；规则较长、需要解释或复用时，应使用普通函数。

函数式工具是表达手段，不要求把所有循环都改成 map 或 lambda。以清楚、便于修改为先。

重点 map 管转换，filter 管筛选，reduce 管累计，sorted 管排序。先说清任务，再选工具。

10 闭包、装饰器与偏函数

进阶理解；接触框架时会逐渐用到

原教程 8.2 返回函数、8.3 匿名函数、8.4 装饰器、8.5 偏函数。

闭包让返回的函数保留外部关联

一个内层函数使用了外层函数中的变量，并且在外层调用结束后还需要继续使用这些关联，就涉及**闭包**。

可以把它理解为“处理方法加上它记住的外部环境”。保存的是变量关联，不是自动冻结的一份数据副本。因此循环中创建多个函数时，之后可能读到同一个外层变量的最终值。

需要重新绑定外层函数的变量时使用 nonlocal；只是读取时通常不需要。

装饰器给函数增加一层处理

装饰器接收一个函数，返回处理后的函数或其他可调用对象。常见用途是添加日志、计时、权限检查等共同步骤，让原函数保留主要业务内容。

`@` 是应用装饰器的语法。理解时分清：装饰器通常在函数定义完成时被应用；包装函数通常在以后每次调用时执行。

包装时需要保留什么

包装函数通常要继续接收原调用的参数，把参数传给原函数，再把原结果返回。异常是否继续抛出，也应按原有接口或明确的设计处理。

`functools.wraps` 用来保留名称、文档等信息，方便调试和工具识别。带参数的装饰器还会增加一个“先接收配置”的层次。

偏函数预先填好部分参数

`functools.partial` 为现有函数绑定一部分参数，得到一个更方便调用的新对象。原函数不会因此被修改。

它主要是减少重复传参，不等于重新写了一个功能；部分预设的关键字参数仍可以在调用时覆盖。

重点 闭包保留环境，装饰器增加共同处理，偏函数预先绑定参数。它们都建立在“函数也是对象”这一点上。

易错 省略返回值、没有透传参数、把计时放在定义阶段，都会让包装后的行为偏离原函数。

11 模块、包与虚拟环境

基础优先

原教程第 9 章、9.1—9.2，以及 16.13 venv。

模块和包负责组织代码

一个 `.py` 文件通常可以作为一个**模块**。模块把相关变量、函数和类放在一起，其他文件通过 import 使用它们。

包把多个模块按层级组织起来。常规包用 `__init__.py` 参与初始化；另有不要求这个文件的命名空间包，初学时先熟悉常规包即可。

导入时会发生什么

导入模块时，Python 通常会执行模块顶层代码并建立模块对象，同一进程中后续导入一般会复用缓存。模块里的函数不会仅因为被定义就自动执行其函数体。

`import` 后用模块名访问成员；`from ... import ...` 将指定名字引入当前范围。文件名不要与正在使用的标准库或第三方库重名，否则可能导入错文件。

主入口用来区分启动方式

`__name__` 帮助区分当前文件是被直接启动，还是被别的模块导入。主入口判断可让演示启动、命令行处理等只在直接运行时发生。

这让同一份代码既可以独立运行，也能被别的程序复用。

标准库、第三方库和框架

标准库随 Python 提供；**第三方库**通常需要另外安装；**框架**还会规定应用怎样组织，并在合适的时机调用你写的处理逻辑。

pip 用来安装发行包；安装名与导入名可能不同。应确保 pip 属于当前准备运行程序的那个解释器。

虚拟环境把项目依赖分开

venv 为项目建立相对独立的 Python 环境，避免不同项目的依赖版本互相干扰。激活环境主要改变当前终端查找程序的位置，不是重装或永久更换系统 Python。

依赖清单用于记录项目需要哪些库及版本；复制项目时通常重建环境，不直接搬运整个虚拟环境文件夹。

> **重点** “已经安装但导入失败”时，先检查安装和运行用的是不是同一个解释器、同一个虚拟环境。

12 类、对象与面向对象思路

基础优先

原教程第 10 章、10.1—10.5。

把相关数据和操作放在一起

类 class 描述一类对象具有什么数据和行为；**实例 instance** 是按这个类创建出的具体对象。**属性**保存状态，**方法**表示这个对象可以完成的操作。

这种组织方式适合描述有多项状态、需要持续交互的数据。简单处理并不一定都要写成类，函数同样有价值。

self 和初始化方法

`self` 按惯例表示当前实例。实例方法通过它读取和修改当前对象的属性；调用普通实例方法时，Python 会自动传入对应实例。

`__init__` 用于初始化实例的状态，应确保对象一建立就具备必要信息。它不负责把业务计算结果当作构造结果返回。

实例属性与类属性

实例属性属于具体实例，各实例通常各有一份；**类属性**属于类，可以被多个实例读取。实例上建立同名属性，可能遮蔽从类上读取到的属性。

需要各实例独立拥有的列表、字典，不应无意中放成共享的类属性。共享统计与个人状态也要分清。

继承、多态与鸭子类型

继承让子类复用或扩展父类的行为；**重写**让子类提供自己的方法实现。**多态**强调：相同的调用接口，可以由不同对象完成各自的行为。

鸭子类型关注对象是否提供所需行为，不强制要求它一定继承某个指定父类。读代码时，应关注方法约定，而不只关注类名。

封装和对象信息

封装是把内部状态与对外接口分开，通过明确方法维护数据规则。单下划线通常表示内部使用约定；双下划线开头可触发名字改写，但不构成不可绕过的安全权限。

`type()` 查看具体类型；`isinstance()` 判断类型关系并考虑继承；`getattr()`、`setattr()` 等按属性名访问对象。

重点 类描述规则，实例保存具体状态；使用对象时看接口，维护对象时守住数据规则。

13 面向对象的高级工具

进阶理解；初学后端前不必全部掌握

原教程第 11 章、11.1—11.6。

属性接口与属性限制

property 让对外看起来像普通属性的读取或赋值，内部实际经过方法处理。这样可以统一校验或即时计算，同时保持使用方式简洁。没有 setter 的 property 通常不允许通过该属性直接赋值。

`__slots__` 可限制常规实例动态新增的属性，并可能节省内存；具体效果受继承结构等影响。它主要是结构和内存方面的工具，不是权限控制。

定制方法对应通用操作

方法	用途
`__str__` `__repr__`	控制面向用户或调试时的文字表示
`__len__`	支持取得对象长度
`__iter__` `__next__`	支持逐项读取及结束迭代
`__getitem__`	支持通过索引或键读取内容
`__getattr__`	常规查找失败后，尝试处理属性访问
`__call__`	让实例能够像函数一样被调用

这些双下划线方法是 Python 的约定接口。实现它们时，要遵守应有返回类型和异常行为；无法处理的属性通常应抛出 AttributeError，而不是随意返回 None。

多重继承与枚举

多重继承允许一个类继承多个父类，方法按 MRO，也就是方法查找顺序寻找。**MixIn** 通常提供某一小块附加能力；`super()` 按继承关系继续协作调用。

Enum 枚举把有限的命名选项组织成明确类型。成员的 name 是名字，value 是值；枚举成员不应简单等同于它内部存放的普通数值。

元类先理解作用即可

类本身也是对象，`type` 可以创建类；**元类 metaclass** 用于参与和定制类的创建过程。它常见于框架底层或映射工具，不是普通业务代码的入门前提。

重点 优先理解 property、继承和常见定制方法。元类先知道“它能改变类怎样创建”，以后读到相关源码再深入。

14 异常、调试与测试

基础优先

原教程第 12 章、12.1—12.4。

三件事分别解决什么问题

异常处理决定出问题后程序怎样响应；**调试**帮助你找出问题发生在哪里；**测试**重复检查程序是否符合预期。它们互相配合，不能彼此替代。

语法错误通常意味着代码结构不合法；运行中的异常可能来自类型、取值、缺失文件、网络等；逻辑错误则可能不报错却得到错误结果。

异常处理的组成

关键字	负责什么
try	放可能失败、需要处理的操作
except	处理指定类型的异常
else	try 正常完成时执行
finally	用于离开这段流程时的清理，正常情况和异常情况都会执行
raise	主动抛出异常，或继续抛出正在处理的异常

异常可以沿调用链向上交给调用者处理。应只捕获自己知道如何处理的异常；不要用一个宽泛的处理块把所有问题悄悄忽略。

怎样读报错信息

traceback 记录异常经过的调用位置。先看末尾的异常类型和说明，再回到相关文件与行号，核对当时的数据与调用路径。

日志 logging 保存程序运行信息；**断点**让程序暂停，便于查看变量和逐步执行。`assert` 可检查开发时的不变量，但可能被优化模式关闭，不适合充当唯一的业务输入校验。

测试关注哪些情况

单元测试检查一小块独立行为，常覆盖正常输入、边界输入、非法输入；unittest 是标准库工具。**文档测试 doctest** 核对文档字符串中的交互式描述与运行行为是否一致。

测试通过，只能说明已检查的情况符合预期，不能证明所有情况永远正确。测试与业务数据应尽量隔离。

重点 先定位原因，再决定如何处理。清理资源放在可靠的退出流程中；避免在 finally 中返回而覆盖原有结果或异常。

15 文件、路径与序列化

基础优先

原教程第 13 章、13.1—13.4。

文件操作的基本过程

IO 指输入和输出，包括读写文件、网络等。文件操作通常经历打开、读写、关闭。`with` 配合文件对象，会在离开相应区域时处理关闭，减少忘记释放资源的问题。

文本模式处理 str，要关心编码；**二进制模式**处理 bytes，适合图片等原始数据。二进制模式不设置文本 encoding。

模式	含义
r	读取已有文件，文件不存在通常报错
w	写入；已有文件会先被截断，不存在则创建
a	在文件末尾追加，不存在时通常创建
带 b	按二进制字节读写

一次全读还是逐步读取

`read()` 可以读取全部或指定数量的内容；`readline()` 读取一行；逐行遍历适合较大的文本。文件有当前位置，已经读过的内容不会每次自动从头开始。

`StringIO` 把文本流放在内存里；`BytesIO` 把字节流放在内存里。它们适合需要文件式接口、又不必真的写入磁盘的处理。

路径和目录

绝对路径从文件系统根位置描述文件；**相对路径**相对于当前工作目录解释。工作目录由启动方式决定，不一定就是脚本所在文件夹。

`os.path`、`pathlib` 用于组织和检查路径；`os.listdir()` 查看目录项，`os.walk()` 递归遍历。重命名、删除、覆盖都是实际修改文件的操作，要确认目标。

序列化是把数据变成可保存的形式

JSON 适合交换常见数据结构，跨语言使用方便；自定义对象通常要先转换成普通结构。`dumps/loads` 主要在文本与对象之间转换，`dump/load` 面向文件。

`ensure_ascii` 控制 JSON 文本是否转义非 ASCII 字符，不决定文件保存的实际字节编码。**pickle** 能保存更多 Python 专有结构，但不适合处理不可信来源的数据。

重点 数据在内存里、保存为文本、编码为字节，是三个相关但不同的层次。先认清当前拿到的是哪一种。

16 进程、线程与并发

进阶理解；先理解场景，再学具体接口

原教程第 14 章、14.1—14.5。

并发与并行的区别

并发关注多个任务都在推进，可以交替执行；**并行**强调多个任务在同一时刻实际执行。任务看起来一起进行，不代表一定使用了多个 CPU 核心。

引入多任务，是为了合理利用等待时间或计算资源，不是任务越多就一定越快。

方式	通俗理解	主要需要关心
进程	相对独立的程序执行单元	创建成本、数据传递、资源使用
线程	同一进程中的执行路线	共享数据、竞争、同步
协程	任务在合适位置主动暂停与恢复	等待点、事件循环、阻塞操作

启动、等待和任务池

任务对象的 `start()` 用来启动新任务，`join()` 用来等待结束；直接调用它的普通运行方法，不等于新建线程或进程。

任务池 Pool 复用一批工作进程或线程，避免无限创建。进程间常通过 Queue 等通信；跨平台多进程脚本要正确组织主入口，工作函数要能被子进程导入。

`subprocess` 用于启动外部程序，与给当前业务函数分配线程或进程并不是同一个层面的操作。

共享状态为什么需要锁

多个执行流程可能在中途交错访问同一份数据。**锁 Lock** 让某段关键操作在同一时刻只由一个执行流程进入，保护需要一起完成的读改写。

锁的范围过大会降低并发度；互相等待错误可能导致死锁。ThreadLocal 为不同线程分别保存关联数据，但不负责让普通共享变量自动安全。

按任务特点选择

CPU 密集型主要忙于计算；**IO 密集型**主要等文件、网络等。在常见带 GIL 的 CPython 中，纯 Python 计算并行通常考虑多进程；等待较多时，可考虑线程或异步。

也有可选的自由线程构建，不能把某一种 GIL 结论当作所有 Python 实现的固定规律。分布式任务还涉及跨机器通信、超时、失败恢复与结果对应，Manager 代理只是其中一种工具。

重点 是否共享数据、主要忙计算还是忙等待、能否承受创建与通信成本，比“用了几条线程”更重要。

17 正则表达式

常用工具；先会读基本符号

原教程第 15 章：正则表达式。

正则表达式描述文字的规则

正则表达式用一套模式描述“什么样的文字符合要求”。常用于查找、提取、替换、按模式拆分，以及验证受限制的文本格式。

它适合清楚的文字模式；复杂嵌套数据和完整协议，通常更适合使用专门解析器。

符号	含义
\d、\w、\s	数字、单词字符、空白字符类别
.	通常匹配换行以外的一个字符
[]	指定字符集合或范围
()	分组，便于提取或组合规则
*、+、?	分别表示零次以上、一次以上、零次或一次
{m,n}	指定重复次数范围
|	表示候选关系
^、$	表示位置边界，具体受模式和选项影响

查找范围要选对

match 从字符串开头尝试匹配，不自动要求全部内容都符合；**search** 在字符串中寻找匹配位置；**fullmatch** 要求整个字符串都符合模式，适合严格的整体格式检查。

group 取得分组内容；**split** 按模式拆分文字；**compile** 提前建立可复用的模式对象。

贪婪与非贪婪

量词通常尽可能多地匹配，但仍要满足完整模式，这叫**贪婪**。量词后加问号通常改成尽可能少地匹配，也就是**非贪婪**。

“少匹配”并不等于不考虑后续规则。读模式时，应一起看字符、次数、分组和边界。

反斜杠有两个解释层次

Python 字符串本身会处理转义，正则引擎也会解释反斜杠。原始字符串可以减少前一层的干扰，让模式更易读，但不会关闭正则表达式自己的规则。

重点 先确认要找局部内容，还是验证整个文本，再选 search、match 或 fullmatch。简化的邮箱规则不等于完整邮箱标准。

18 时间、数据工具与命令行参数

按需要查用

原教程 16.1 datetime、16.2 collections、16.3 argparse。

时间不只有“显示出来的几点”

datetime 表示日期时间；**timedelta** 表示时间差；**timestamp** 用统一起点以来的秒数表示时刻。显示时区可以不同，但同一个时刻对应的时间戳不因显示方式改变。

naive 时间没有时区信息；**aware 时间**带有时区信息。给一个钟面时间附加时区，是在解释它；转换时区则要保持同一时刻，换一种当地显示。

`strptime` 把文字解析成时间，`strftime` 把时间按格式写成文字。固定 UTC 偏移不能代替包含夏令时规则的地区时区；地区时区可使用 zoneinfo。

collections 提供更合适的数据容器

工具	主要用途
namedtuple	用字段名读取固定结构的元组数据
deque	高效地从两端加入或取出元素
defaultdict	访问缺失键时按规则生成默认值
OrderedDict	除保持顺序外，还提供移动次序等专门操作
ChainMap	按优先顺序联合查看多份映射
Counter	统计元素出现次数

普通 dict 已保留插入顺序，不必为了这一点专门换成 OrderedDict。defaultdict 的缺失键机制，通常由索引访问触发，不能简单套到 get 等所有读取方式上。

argparse 帮脚本理解启动参数

命令行参数是在启动程序时传入的配置或数据。argparse 统一负责声明参数名、必填项、默认值、类型转换和帮助信息。

位置参数按出现顺序对应；**选项参数**常带有前缀。布尔开关通常用专门的 action 表达，而不是把任意非空字符串交给 bool 转换。

重点 这些模块主要用于减少重复造工具。先记住各自解决的问题，需要时再查具体方法和参数。

19 二进制、摘要与资源管理

按需要认识；密码相关概念要分清

原教程 16.4—16.9：base64、struct、hashlib、hmac、itertools、contextlib。

Base64 与二进制布局

Base64 把字节编码成适合文本环境传递的字符形式，末尾的等号可能用于填充。它可以解码还原，**不是加密**；URL 安全版本使用适合 URL 的字符。

struct 按约定的长度、类型和字节顺序，把数值与二进制数据互相转换。**大端、小端**是多字节数值的排列顺序；读取时必须与数据格式的约定一致。

摘要、盐和 HMAC

哈希摘要把数据映射成固定规则的摘要值，用于比较和完整性检查；它不是可以解密还原内容的加密。只拿到未经可信验证的摘要，不能单独证明数据来自谁。

盐 salt 是与密码计算一起使用的额外数据；每条记录使用独立随机盐，可减少相同密码的记录重复和预计算复用。盐通常不要求保密。

HMAC 把密钥与消息一起用于认证码计算，强调带密钥的消息认证。比较认证码时，可使用专门的比较接口。

> 易错　直接使用快速 MD5、普通 SHA 摘要或单轮 HMAC，不等于现代密码存储方案。密码应使用专门、可调计算成本的派生算法，不能只记住“加盐就安全”。

itertools 组合迭代过程

`count` 连续计数，`cycle` 循环输入，`repeat` 重复数据；它们可能形成无限序列，要先想清楚如何结束。

`islice` 限定读取范围，`chain` 连接多个输入，`takewhile` 读取开头连续满足条件的一段。`groupby` 只把**相邻且键相同**的元素分组，不会自动完成全局分组。

contextlib 管理进入与退出

上下文管理把资源准备、使用和退出清理组织在一起。`contextmanager` 可用单次 yield 的函数组织这三个阶段；`closing` 可在退出时调用对象的 close。

关键不是把写法变短，而是确保正常完成和出现异常时，都有明确的清理路径。

> 重点　Base64 管表示形式，struct 管字节布局，hashlib 管摘要，hmac 管带密钥认证；不要把它们都当作“加密”。

20 获取网页与解析数据

应用认识；理解输入和输出类型

原教程 16.10 urllib、16.11 XML、16.12 HTMLParser。

获取数据与理解数据是两步

HTTP 客户端负责发送请求、取得响应；**解析器**负责把响应内容变成程序可使用的结构。请求成功，并不意味着内容一定是想要的 JSON 或网页。

处理时通常先看响应状态和内容类型，再按合适的字符编码理解文本，最后选择 JSON、XML、HTML 等对应的解析方法。

urllib 做什么

`urllib.request` 用于网络请求；查询参数需要正确的 URL 编码，不能把任意文字随意拼接到地址里。响应体首先是字节数据；响应头可能说明文字编码等信息。

请求应有超时设置。网络不可用、服务拒绝、认证失效、返回格式变化，都应作为正常需要处理的失败情况考虑。

三种常见内容形式

形式	核心用途	常见处理方式
JSON	交换结构化数据	转成字典、列表等 Python 数据
XML	用标签表达层级结构	按树结构读取，或响应解析事件
HTML	描述网页内容结构	结合标签、属性和嵌套位置提取内容

DOM、SAX 与事件回调

DOM 把整个 XML 文档组织成树，方便反复访问，但会使用相应内存。**SAX/Expat** 随着读取过程报告标签和文本事件，适合逐步处理。

回调函数是你先交给工具、由工具在合适时机调用的函数。文本回调可能分成多次发生，需要累计内容，不能默认一次回调就是一段完整文字。

HTMLParser 需要记录当前位置

HTMLParser 会通知你遇到了开始标签、文本或结束标签。提取信息时，要跟踪正在读取哪个区域、文字属于哪个字段；网页结构变化时，提取规则可能需要调整。

重点 先取到字节，再按编码理解文字，再按格式解析结构。JSON 文本与解析后的字典不是同一种对象。

21 第三方库与图形界面

按项目需要选择，不必全部提前学完

原教程第 17 章、17.1—17.4；第 18 章、18.1 海龟绘图。

常用第三方库分别负责什么

库	主要用途	使用时的关键点
Pillow	打开、缩放、处理和保存图像	关注尺寸、颜色模式、输出格式
requests	更方便地发送 HTTP 请求	关注超时、状态码、返回格式
chardet	推测字节数据可能采用的编码	推测带有不确定性，不是格式保证
psutil	查看进程和系统资源状态	状态会变化，部分信息需要权限

安装 Pillow 后，导入时使用的名字是 PIL。安装名与导入名不一致，并不一定表示安装错误。

图像与网络处理中的常见区别

Pillow 中保持比例的缩略和指定宽高的缩放不是同一要求；有的操作修改原对象，有的返回新对象，要按接口约定使用。图像能否保存为某格式，也与颜色模式有关。

requests 中查询参数、表单数据、JSON 请求体各有对应参数。能调用响应的 JSON 解析方法，不代表远端一定返回了合法 JSON；应先结合状态与内容判断。

编码检测的 confidence 表示推断信心，不是正确率承诺。若协议或文件规范已经明确编码，应优先依照明确约定。

图形界面依靠事件循环

Tkinter 是 Python 常用的图形界面接口。窗口、输入框、按钮是控件；点击、输入等动作产生事件，程序通过回调响应。

注册按钮回调时，通常交给界面的是函数对象，让它以后再调用。若在注册时就执行函数，会使执行时机不符合预期。

事件循环负责持续处理事件和重绘界面。长时间运行的操作如果堵住它，窗口就可能显得没有响应。

turtle 用运动描述图形

海龟绘图 turtle 用当前位置、方向、前进、转向等指令组织图形，适合理解流程和绘图。它通常需要可用的桌面与 Tk 环境，不是所有命令行环境都能显示窗口。

重点 不需要背下每个库的全部接口。先记住它解决什么任务，再学习当前项目真正用到的部分。

22 网络通信与电子邮件

应用认识

原教程第 19 章、19.1—19.3；第 20 章、20.1—20.2。

地址、端口与 socket

IP 地址用于定位网络中的主机；**端口**区分主机上的服务入口；**socket** 是程序收发网络数据的接口。

客户端通常发起连接或请求，服务端在指定入口等待并处理。实际收发的数据是字节，需要明确编码或二进制格式。

方式	特点	程序需要考虑
TCP	按顺序提供可靠字节流	自己规定一条消息从哪里结束
UDP	按数据报发送，不保证到达与顺序	是否需要超时、重试、去重等机制

TCP 不保留应用消息边界

一次发送不保证对应一次接收，数据可能被分开到达，也可能与其他数据一起到达。应用通常通过固定长度、分隔符或长度前缀区分消息。

TCP 服务端的监听 socket 用于接入连接，接入后通常通过另一个连接对象与客户端通信。接收返回空字节，通常表示对方已关闭连接。

邮件协议各管一部分

SMTP 主要负责提交和发送邮件；**POP3** 主要负责下载邮箱中的邮件；**IMAP** 侧重服务器上的邮箱管理和同步，原教程重点讲 SMTP 与 POP3。

邮件头里的显示收件人与传输过程中的实际收件地址属于不同层次，不能只修改文字显示就认为发送目标也一定改变。

MIME 让邮件包含不同内容

MIME 描述邮件内容类型和组合方式，支持纯文本、HTML、附件与内嵌图片。多部分邮件需要按各部分的类型和编码读取，不能把全部内容当普通文本。

安全连接使用 TLS 等保护传输；服务器地址、端口和登录方式由实际邮件服务决定。程序中的账户认证与消息内容构造，也应分开理解。

重点 TCP 管字节流传输，SMTP/POP3 管邮件收发流程，MIME 管邮件内部怎样组织内容。

23 数据库、SQL 与 ORM

后端基础优先

原教程第 21 章、21.1 SQLite、21.2 MySQL、21.3 SQLAlchemy。

数据库负责长期保存并组织数据

关系数据库主要用表组织数据：行是记录，列是字段，主键用于标识记录。表之间可以建立关联。**NoSQL** 是多种非关系存储的统称，不等于没有查询能力。

SQLite 通常直接使用本地数据库文件，适合起步学习和轻量应用；**MySQL** 作为独立服务运行，客户端通过连接访问。两者不是同一种部署方式。

SQL 描述对数据的操作

操作	用途
SELECT	查询数据
INSERT	新增记录
UPDATE	修改已有记录
DELETE	删除记录
WHERE	限定要处理哪些记录
ORDER BY	明确结果的排序方式

未明确排序时，不应依赖某种固定返回顺序。修改和删除前尤其要确认限制条件，避免处理范围过大。

连接、游标与事务

连接 Connection 维持与数据库的交互；**游标 Cursor** 执行语句、读取结果。**事务 Transaction** 把一组操作组织成一个提交或撤销的单位。

commit 提交让当前事务的修改正式生效；**rollback 回滚**撤销尚未提交的修改。关闭连接是释放资源，不等于已经正确提交了数据。

参数化查询与 ORM

参数化查询把 SQL 语句结构和参数值分开交给驱动，避免把用户输入直接当作 SQL 结构拼接。不同驱动的占位形式可能不同。

ORM 对象关系映射让程序通过类、对象和属性操作表、行和列。SQLAlchemy 提供相关工具；Engine 管理连接基础设施，Session 跟踪对象变化与事务工作。使用 ORM 后，仍需要理解数据关系、查询和事务。

重点 后端入门先掌握“新增、查询、修改、删除”和事务。ORM 提高表达便利，不会让数据库原理消失。

24 HTTP、网页与服务端接口

后端基础优先

原教程第 22 章、22.1 HTTP、22.2 HTML、22.3 WSGI。

一次访问经过哪些环节

客户端发请求 → 服务端找到处理逻辑 → 读取或修改数据 → 服务端返回响应 → 客户端显示或继续处理。浏览器是客户端的一种，其他程序也可以调用后端。

前端主要处理用户看到的界面和交互；**后端**主要负责业务规则、数据访问和权限等。两者可以通过 HTTP 协作。

请求与响应分别有什么

请求通常包括方法、地址或路径、请求头，以及可能存在的请求体。**路由**把方法和路径对应到处理逻辑。**接口 API** 是约定好的调用入口，说明如何传入数据、会返回什么。

响应通常包括状态码、响应头和响应体。状态码说明处理结果；响应体可以是 HTML、JSON、文件或其他数据。

名词	主要作用
GET	通常用于获取资源
POST	通常用于提交数据或请求处理
HTTPS	在 HTTP 通信中使用 TLS 保护传输
JSON	常用于客户端与后端交换结构化数据

GET 与 POST 的选择不能替代传输加密。接口语义、身份认证和权限检查也属于不同层面的问题。

HTML、CSS、JavaScript 与模板

HTML 描述网页结构，**CSS** 控制样式，**JavaScript** 处理浏览器端行为。**模板**把页面结构与要填入的数据分开，由服务端组合成页面。

用户提供的文字放入 HTML 时要正确转义，防止被当作页面代码解释。表单、JSON 请求体、网址参数是不同的数据来源，读取方式也应对应。

WSGI 规定服务器怎样调用 Python 应用

WSGI 是 Web 服务器与 Python 应用之间的一套接口约定，负责把请求交给应用，并接收响应结果。它不是 HTTP 协议本身，也不是某个完整框架。

网站的业务逻辑、框架、服务进程和网络协议各有职责，初学时先顺着一次请求理解它们的衔接。

重点 后端并不是“写网页文字”。重点是：接收数据、判断规则、访问存储、返回约定结果。

25 后端框架应该怎样理解

入门认识；结合你目前只会 Python 基础的情况补充

原教程 22.4 使用 Web 框架、22.5 使用模板；框架比较为本次补充说明。

框架让你专注自己的处理逻辑

后端应用经常都要做路由、读取请求、生成响应、处理错误和组织配置。**框架**把这些重复工作组织好，让你在规定的位置填写自己的业务逻辑。

学习框架不是换一门语言，函数、类、字典、异常、模块等 Python 基础仍然在使用，只是被放到一个有约定的应用结构中。

框架	主要特点	常见使用方向
Flask	核心轻量，许多功能按需组合	理解基本请求流程，建立小网站和服务
Django	自带较完整的数据操作、认证、管理后台等配套	内容网站、管理系统及完整业务应用
FastAPI	围绕接口开发，强调类型校验与接口文档，支持异步	为网页、移动端或其他程序提供数据接口

这些方向不是严格边界，各框架的能力有重叠。选择主要看应用需求、团队经验和学习目标；不能只根据名字或某项速度宣传下结论。

几个常见后端名词

视图或处理函数接收框架交来的请求信息并返回结果。**中间件**在请求处理前后执行公共逻辑。**Cookie** 是浏览器保存并随请求携带的数据；**Session 会话机制**用于关联多次请求之间的用户状态，具体存储方式由实现决定。

认证判断是谁，**授权**判断允许做什么。已经登录不代表可以读取或修改所有人的数据。

当前阶段优先理解什么

先理解路由、请求参数、响应、数据库和登录状态这条主线，再逐步认识应用结构、测试与部署。装饰器等语法可以结合框架的实际用法理解，不必先掌握元类或复杂异步。

本笔记沿教程保留 Flask 相关知识。框架负责组织流程，业务函数仍要自己把输入、处理规则和返回值说清楚。

重点 先学一个框架的基本流程。能够解释“一次请求怎样变成一个结果”，比一次记住三个框架的接口更重要。

26 协程、asyncio 与 aiohttp

进阶理解；后端起步时可以稍后深入

原教程第 23 章、23.1 协程、23.2 asyncio、23.3 aiohttp。

异步主要利用等待的时间

程序等待网络或其他 IO 时，可以暂时让出执行机会，让其他已经准备好的任务继续推进。**异步**把这种等待与继续的安排组织起来，不是把所有操作都变成更快的计算。

事件循环 Event Loop 负责查看哪些任务可以继续、哪些还在等待。**协程 Coroutine** 是可以在指定位置暂停和恢复的执行过程。

async 与 await 各做什么

`async def` 定义协程函数；调用它通常得到协程对象，不会因此自动跑完或自动并发。`await` 用于等待可等待对象，在确实需要等待时允许调度其他工作。

`asyncio.run()` 运行顶层异步入口；在已有事件循环的环境中，不能随意再启动一个嵌套入口，应按环境约定调用。

顺序等待与并发调度

逐个等待一个任务结束，再开始下一个，通常仍是顺序流程。创建多个 Task 或使用 gather 等工具，可以把多个任务交给事件循环安排。

gather 收集结果时，通常按传入任务的顺序排列，不是按完成顺序排列。任务数量仍需限制，否则会消耗过多连接和内存。

生成器协作与现代协程的关系

生成器的 yield 和 send 能演示暂停、传入数据、恢复执行的过程；现代 asyncio 主要使用 async/await 表达异步逻辑。普通生成器不等于已经接入了异步网络框架。

aiohttp 是在 asyncio 基础上处理 HTTP 的库，可用于异步客户端和服务器。`async with` 用于带有异步进入或退出操作的资源管理。

阻塞操作仍然会堵住流程

在协程里直接调用阻塞的睡眠、同步网络请求或长时间计算，仍可能阻塞事件循环。应使用合适的异步接口，或把适合的阻塞任务交给线程等机制。

同一线程里的多个协程也会在等待点交错访问数据，仍可能需要异步锁等协调方式。单线程不等于任何共享状态都不会出错。

重点 async 是一种组织方式，不是自动加速开关。先看任务如何被调度、在哪里等待、有没有阻塞整个事件循环。

27 常用排查信息与概念速查

基础复习

原教程第 24 章 FAQ、第 25 章期末总结；结合前文归纳。

路径、参数和解释器信息

当前工作目录是程序解释相对路径时的依据；**模块文件位置**是这份代码实际保存在哪里，两者可能不同。

`os.getcwd()` 提供工作目录；`__file__` 通常提供模块文件位置，但部分交互环境没有它；`sys.argv` 提供启动参数；`sys.executable` 提供当前解释器路径。

遇到找不到文件、装包后仍导入失败等问题，这些信息有助于先确认运行环境，再检查业务代码。

容易混淆的概念放在一起看

两个概念	核心区别
赋值 / 复制	关联名字 / 创建另一份对象结构
相等 / 同一对象	比较内容值 / 比较对象身份
打印 / 返回	显示内容 / 把结果交回调用者
str / bytes	文字 / 字节数据
可迭代对象 / 迭代器	能提供逐项读取 / 保存当前读取进度
模块 / 包	一份代码组织单元 / 按层级组合模块
提交 / 关闭连接	确认事务修改 / 释放连接资源
并发 / 并行	多任务都在推进 / 同时实际执行
认证 / 授权	判断是谁 / 判断能做什么

从基础到应用的主线

先把类型、容器、流程、函数、模块、类和异常理解清楚，再把它们用于文件、数据库和网络。遇到新库时，先辨认它接收什么、返回什么、会改变什么、可能怎样失败。

元类、分布式任务、复杂异步可以先保留概念印象，随着实际需要再深入。它们不是开始学习基本后端流程的前置门槛。

重点 把“数据是什么类型、代码在哪一步执行、状态有没有改变、资源何时释放”串起来，许多零散知识就会变成一条清楚的处理流程。

来源与章节对照

本版依据你原来的全教程笔记重新组织，原教程内容核对日为 2026-11-09，本次修订日为 2026-11-10。保留知识范围，按主题合并重复导言；本版为概念梳理，不替代原教程的完整展开。

本版主题	原教程范围
01	1—4：环境、运行与输入输出
02—05	5：类型、字符串、容器、控制流
06—07	6：函数、参数与递归
08	7：切片、迭代与生成器
09—10	8：函数式编程与装饰器
11	9、16.13：模块、包与虚拟环境
12	10：面向对象基础
13	11：面向对象高级工具
14	12：异常、调试与测试
15	13：文件、目录与序列化
16	14：进程、线程与分布式任务
17	15：正则表达式
18—20	16：常用内建模块
21	17—18：第三方库与图形界面
22	19—20：网络与邮件
23	21：数据库
24—25	22：Web、WSGI、框架与模板
26	23：异步 IO
27	24—25：FAQ 与知识归纳

原教程：廖雪峰《Python 教程》完整目录与正文。

框架补充说明：Flask 官方文档；Django 官方概览；FastAPI 官方文档。

版本细节参考：Python threading；hashlib；asyncio。

原教程作者为廖雪峰。本版为内容重组与通俗改写，不代表原作者提供的官方笔记；依CC BY-NC-SA 4.0共享，保留署名、非商业使用与相同方式共享条件。